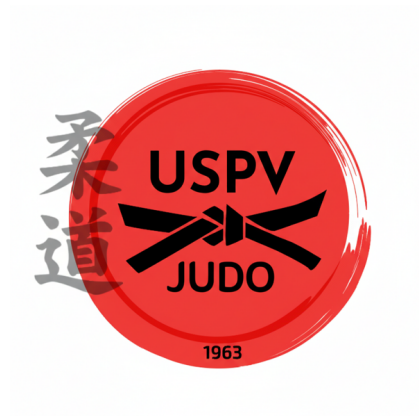




DOCUMENT D'ARCHITECTURE TECHNIQUE (DAT)

Projet de Refonte de l'Application Web
USPV Judo Pont-de-Roide

Document lié	Cahier des charges USPV Judo
Projet	Refonte de l'application web et CRM associatif
Auteur	Mehdi LAFAY
Superviseur technique	David LECOCQ
Date	10 avril 2026
Statut	Document de travail — évolutif pendant le stage

Stage de 2^{ème} année d'informatique

Table des matières

1	OBJET DU DOCUMENT	3
2	VUE D'ENSEMBLE DE L'ARCHITECTURE	3
2.1	Flux principal	3
2.2	Flux d'authentification	3
3	STACK TECHNIQUE RETENUE	4
3.1	Tableau récapitulatif	4
3.2	Justification des choix majeurs	5
3.2.1	Nuxt 3 / Vue.js 3	5
3.2.2	Node.js / Express.js	5
3.2.3	PostgreSQL	6
3.2.4	Driver natif (pg) vs ORM	6
3.2.5	Alternatives écartées	6
4	ARCHITECTURE DÉTAILLÉE	6
4.1	Structure des répertoires (cible)	6
4.2	Modèle de données (MCD fonctionnel)	7
4.3	Routes API (périmètre MVP)	9
4.4	Stratégie de versioning API	10
4.5	Catalogue des codes de réponse HTTP	10
5	SÉCURITÉ	11
5.1	Synthèse des mesures de sécurité	11
5.2	Gestion des tokens JWT	11
5.3	Protection contre les attaques courantes (OWASP)	12
6	STRATÉGIE DE TESTS	12
6.1	Pyramide de tests retenue	12
6.2	Configuration de test	13
6.3	Scénarios prioritaires	13
6.3.1	Tests unitaires	13
6.3.2	Tests d'intégration API	13
6.3.3	Tests E2E (si livrés)	13
6.4	Données de test (Seeding)	14
7	DÉPLOIEMENT ET EXPLOITATION	14
7.1	Architecture serveur cible	14
7.2	Procédure de déploiement (mode nominal — Garanti)	14
7.3	Configuration Nginx (cible)	15
7.4	Variables d'environnement	16
8	SAUVEGARDE ET RESTAURATION	16
8.1	Dispositif de sauvegarde	16
8.2	Script de sauvegarde (cible)	16
8.3	Procédure de restauration	17
8.4	Test de restauration	17
9	JOURNALISATION ET SURVEILLANCE	17
9.1	Bibliothèque de logging	17
9.2	Logs applicatifs (Garanti)	18

9.3	Surveillance (Souhaité / Éventuel)	18
10	INSTRUMENTATION ET MESURE D'AUDIENCE	18
10.1	Choix de l'outil (Souhaité)	18
10.2	Événements applicatifs (Souhaité)	18
11	ENVIRONNEMENTS	19
12	PERFORMANCE ET DIMENSIONNEMENT	19
12.1	Charge attendue	19
12.2	Optimisations prévues	19
13	CI/CD (ÉVENTUEL)	20
14	SYNTHÈSE DES DÉPENDANCES NPM PRINCIPALES	20
15	GLOSSAIRE	21

1 OBJET DU DOCUMENT

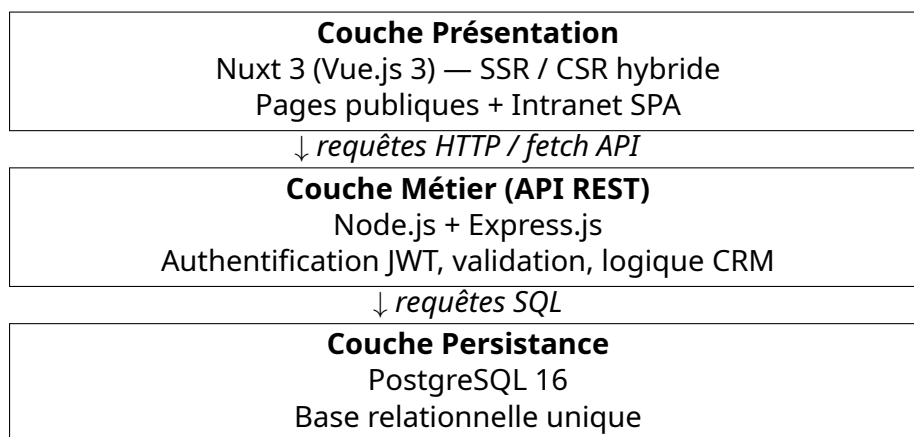
Ce document décrit les choix d'architecture, la stack technique retenue, les outils de développement, les procédures d'exploitation et les paramètres de sécurité de l'application web USPV Judo. Il constitue l'annexe technique du Cahier des charges USPV Judo, auquel il se réfère pour les exigences fonctionnelles, la priorisation et les critères de recette.

Périmètre : ce DAT couvre l'ensemble du périmètre technique du projet, y compris les briques classées **Souhaité** et **Éventuel**. Seule la stack nécessaire au MVP **Garanti** conditionne la livraison ; les outils liés aux briques secondaires sont documentés à titre de cible.

Évolutivité : ce document sera mis à jour au fil du stage si un choix technique évolue.

2 VUE D'ENSEMBLE DE L'ARCHITECTURE

L'application suit une architecture **Full-Stack à trois couches** séparées logiquement :



Principe de séparation : le front-end et l'API sont deux processus distincts communiquant exclusivement via HTTP. Cette séparation permet un développement parallèle, un déploiement indépendant si nécessaire et une surface d'attaque réduite (l'API contrôle tous les accès aux données).

2.1 Flux principal

1. Le navigateur du visiteur charge les pages publiques rendues côté serveur par Nuxt (SSR) pour le SEO.
2. Les interactions dynamiques (formulaire multi-étapes, intranet) sont gérées en mode client (CSR/SPA) avec appels API REST.
3. L'API Express valide chaque requête (authentification, autorisation, validation des données), exécute la logique métier et interroge PostgreSQL.
4. Les réponses JSON sont retournées au front-end pour affichage.

2.2 Flux d'authentification

1. L'administrateur soumet ses identifiants via le formulaire de connexion.
2. L'API vérifie les identifiants contre le hash bcrypt stocké en base.

3. En cas de succès, l'API génère un token JWT signé (durée de vie limitée) et le transmet au navigateur via un cookie `httpOnly`, `secure` et `SameSite=Strict`.
4. Le navigateur envoie automatiquement le cookie d'authentification avec chaque requête vers les routes protégées ; le token n'est jamais accessible au JavaScript client.
5. Un middleware Express extrait le token du cookie, vérifie sa validité et son expiration, puis contrôle le rôle avant tout accès aux ressources privées.

3 STACK TECHNIQUE RETENUE

3.1 Tableau récapitulatif

Couche	Technologie / Outil	Rôle et justification
Front-end (Interface web)		
Framework	Nuxt 3	Meta-framework Vue.js avec rendu hybride SSR/CSR, routing automatique, SEO natif.
Bibliothèque réactive	Vue.js 3 (Composition API)	Composants réactifs, gestion d'état local, écosystème mature.
Gestion d'état	Pinia	Store centralisé pour l'état du formulaire multi-étapes et l'état d'authentification.
Style / CSS	Approche utilitaire ou composant	Le choix précis (CSS natif, framework utilitaire ou bibliothèque de composants) sera arrêté lors de la phase maquettes.
Validation front	Validation native HTML5 + logique Vue	Validation temps réel des champs du formulaire d'inscription.
Accessibilité	HTML sémantique, contrastes, clavier	Exigences de base définies dans le CDC (contraste $\geq 4.5:1$, navigation clavier sur les parcours critiques, attributs <code>alt</code> , structure sémantique).
Back-end (API REST)		
Runtime	Node.js (LTS en vigueur)	Environnement JavaScript côté serveur, léger et adapté aux API REST.
Framework web	Express.js	Framework HTTP minimaliste, écosystème de middlewares riche, courbe d'apprentissage faible.
Authentification	JWT (jsonwebtoken)	Tokens signés sans état côté serveur, adaptés à une API REST stateless.
Hachage mots de passe	bcrypt (bcryptjs)	Algorithme dédié au hachage de mots de passe avec salage automatique.
Validation serveur	express-validator	Assainissement et validation des entrées utilisateur côté API ; protection contre les injections.
Accès BDD	Driver natif (pg)	Requêtes SQL explicites via le driver PostgreSQL pour Node.js ; contrôle total sur les requêtes et les performances.
Base de données		
SGBD	PostgreSQL 16	SGBD relationnel open-source, contraintes d'intégrité, types riches, fiabilité éprouvée.
Sauvegarde	pg_dump	Export logique complet de la base pour les sauvegardes quotidiennes.
Infrastructure et déploiement		
Serveur	VPS Linux (Ubuntu Server LTS)	Serveur privé virtuel avec contrôle total sur l'environnement.

Hébergeur cible	OVHcloud / Scaleway / Hostinger	Prestataires compatibles budget associatif (4-6 €/mois).
Certificat TLS	Let's Encrypt (renouvellement auto)	HTTPS obligatoire en production, gratuit et automatisé.
Reverse proxy	Nginx	Terminaison TLS, proxy vers l'API et le serveur Nuxt, compression, en-têtes de sécurité.
Gestion de processus	PM2 ou systemd	Redémarrage automatique des processus Node.js en cas de crash, gestion des logs.
Transfert sauvegardes	rclone (chiffré)	Externalisation des dumps PostgreSQL vers un stockage objet distant.
Tâches planifiées	cron	Déclenchement des sauvegardes nocturnes et des tâches de maintenance.

Outillage de développement

Gestionnaire de paquets	npm	Gestion des dépendances front-end et back-end.
Versionnement	Git (+ dépôt distant)	Suivi des modifications, branches, historique.
Tests unitaires / intégration	Vitest	Framework de test rapide, compatible Vue/Nuxt et Node.js.
Tests composants Vue	@vue/test-utils	Montage virtuel des composants pour tests isolés.
Tests API HTTP	Supertest	Simulation d'appels HTTP contre l'API Express.
Couverture de code	@vitest/coverage-v8	Mesure de la couverture de code sur les modules critiques.
Tests E2E (si livrés)	Playwright ou Cypress	Simulation navigateur pour les parcours utilisateur complets.

CI/CD (Éventuel)

Pipeline	GitHub Actions	Build, tests et déploiement automatisés — hors scope MVP.
----------	----------------	---

3.2 Justification des choix majeurs

3.2.1 Nuxt 3 / Vue.js 3

- **SEO natif** : Le mode SSR (Server-Side Rendering) de Nuxt génère le HTML des pages publiques côté serveur, les rendant immédiatement lisibles par les robots de Google sans JavaScript côté client.
- **Hybridation SSR/CSR** : Les pages publiques (accueil, infos, inscriptions) bénéficient du rendu serveur pour le référencement ; l'intranet d'administration fonctionne en mode SPA (Single Page Application) pour la réactivité.
- **Structure de projet claire** : Routing automatique basé sur le système de fichiers, layouts réutilisables, middlewares intégrés.
- **Objectif pédagogique** : Maîtrise d'un framework Full-Stack JavaScript moderne, cohérent avec la formation.

3.2.2 Node.js / Express.js

- **Cohérence de langage** : JavaScript de bout en bout (front + back) réduit la charge cognitive et les erreurs de contexte.
- **Express minimaliste** : Framework léger et explicite, pas de magie ; chaque middleware est visible et compréhensible.

- **Écosystème riche** : Middlewares communautaires éprouvés (cors, helmet, express-rate-limit, express-validator).
- **Adapté au stage** : Courbe d'apprentissage maîtrisable dans le temps imparti, documentation abondante.

3.2.3 PostgreSQL

- **Intégrité des données** : Contraintes de clé étrangère, types de données stricts, transactions ACID — essentiels pour des données de mineurs.
- **Open-source et gratuit** : Aucun coût de licence, communauté active.
- **Outils de sauvegarde** : `pg_dump` pour les exports logiques, restauration simple et testable.
- **Évolutivité** : Supporte sans problème le volume attendu (quelques centaines de dossiers par saison).

3.2.4 Driver natif (pg) vs ORM

- **Choix retenu** : Le driver natif `pg` (node-postgres) est privilégié pour le MVP afin de garder un contrôle total sur les requêtes SQL, faciliter le débogage et éviter la couche d'abstraction d'un ORM (Prisma, Sequelize) qui peut complexifier le projet pour un gain limité à cette échelle.
- **Requêtes paramétrées** : Toutes les requêtes utilisent des paramètres positionnels (`$1`, `$2`, ...) pour prévenir les injections SQL. Aucune concaténation de chaîne utilisateur dans les requêtes.
- **Évolution possible** : Si le projet évolue significativement en post-MVP (modèle de données complexe, migrations fréquentes), l'introduction d'un ORM léger pourra être envisagée.

3.2.5 Alternatives écartées

- **Firebase / Supabase (BaaS)** : Solutions rapides à démarrer mais limitant le contrôle sur la logique serveur, la structure de la base et les politiques de sécurité personnalisées. Incompatibles avec l'objectif pédagogique de maîtrise Full-Stack.
- **Next.js / React** : Écosystème équivalent mais Vue.js a été retenu pour sa syntaxe plus lisible, son approche progressive et la familiarité de l'équipe.
- **NestJS** : Framework back-end structuré (TypeScript, injection de dépendances) jugé surdimensionné pour le périmètre MVP et le temps de stage disponible.

4 ARCHITECTURE DÉTAILLÉE

4.1 Structure des répertoires (cible)

```
uspv-judo/
  client/
    pages/
      index.vue
      club.vue
      infos.vue
```

Application Nuxt 3
Routes automatiques
Page d'accueil
Présentation du club
Horaires et tarifs

```

    inscription.vue    # Tunnel d'inscription
    contact.vue        # Page de contact
    boutique.vue       # Catalogue (si livré)
    evenements.vue     # Calendrier/résultats (si livré)
    login.vue          # Connexion admin
    admin/             # Intranet (SPA protégée)
        index.vue     # Tableau de bord
        dossier/
            [id].vue   # Fiche dossier
    components/        # Composants réutilisables
    composables/        # Logique partagée (useAuth, etc.)
    stores/            # Stores Pinia
    layouts/           # Layouts (public, admin)
    middleware/        # Guards de navigation
    assets/            # Styles, images statiques
    public/            # Fichiers statiques (favicon, etc.)
    nuxt.config.ts      # Configuration Nuxt
api/                  # API Express
    src/
        index.js      # Point d'entrée Express
        config/        # Configuration (env, DB)
        routes/        # Définition des routes
        controllers/   # Logique métier par domaine
        middlewares/   # Auth, validation, rate-limit
        models/        # Accès données (requêtes SQL)
        utils/         # Helpers (hachage, JWT, etc.)
    tests/            # Tests unitaires et intégration
    package.json
    .env.example      # Template des variables d'env
database/
    schema.sql        # Script de création des tables
    seed.sql          # Données fictives de test
    migrations/       # Scripts de migration (si besoin)
docs/
    dat.pdf           # Documentation projet
    manuel-utilisateur.pdf

```

Note : l'arborescence ci-dessus est une cible de conception. Le sous-répertoire `api/src/` pourra être aplati (fichiers directement dans `api/`) selon l'organisation retenue pendant le développement ; la structure livrée sera documentée dans le README du dépôt.

4.2 Modèle de données (MCD fonctionnel)

Le schéma fonctionnel défini dans le cahier des charges (section « Spécification fonctionnelle du CRM associatif ») se décline techniquement comme suit. Le MLD complet sera produit pendant la phase 2 (Architecture et Réalisation).

Table	Colonnes clés	Contraintes principales
dossier	id (PK, serial), statut (enum), date_soumission (timestampz), note (text, nullable), enfant_id (FK), representant_id (FK)	statut IN ('en_attente', 'incomplet', 'valide', 'rejeté'); NOT NULL sur statut et date_soumission. Le champ note sert à la fois de motif de rejet et de note de suivi (ex. : pièce manquante en cas de statut <i>Incomplet</i>).
enfant	id (PK, serial), nom (varchar 100), prenom (varchar 100), date_naissance (date), categorie (varchar 50), discipline (varchar 20, DEFAULT 'judo'), classe (varchar 50, nullable)	NOT NULL sur nom, prenom, date_naissance. discipline IN ('judo', 'jiu-jitsu').
representant	id (PK, serial), nom (varchar 100), prenom (varchar 100), email (varchar 255), telephone (varchar 20), adresse (text)	NOT NULL sur nom, prenom, email, telephone. Email indexé pour le regroupement fratrie.
trace_legale	id (PK, serial), dossier_id (FK), type_trace (varchar 50), version_document (varchar 50), date_validation (timestampz)	FK vers dossier ; NOT NULL sur tous les champs.
paiement	id (PK, serial), dossier_id (FK), statut_reglement (enum), mode (varchar 50, nullable), date_reception (date, nullable)	statut IN ('non_recu', 'partiel', 'recu').
historique	id (PK, serial), dossier_id (FK), action (varchar 100), auteur (varchar 255), horodatage (timestampz)	Insertion seule (append-only), pas de mise à jour.
utilisateur	id (PK, serial), email (varchar 255, unique), mot_de_passe_hash (varchar 255), role (varchar 20), actif (boolean)	role IN ('admin') pour le MVP ; mot de passe haché bcrypt.
<i>Tables — Souhaité (si module boutique livré)</i>		
article	id (PK, serial), nom (varchar 150), description (text, nullable), prix (numeric 8,2), image_path (varchar 255, nullable), actif (boolean DEFAULT true), date_creation (timestampz DEFAULT now())	NOT NULL sur nom et prix ; prix > 0.

Notes de conception :

- **État « Brouillon »** : L'état *Brouillon* d'un formulaire en cours de saisie reste côté client (store Pinia) et n'est pas persisté en base de données ; seule la soumission crée un enregistrement serveur.
- **Contrainte sur enfant.categorie** : Une contrainte CHECK pourra être ajoutée pour limiter les valeurs aux catégories officielles FFJDA (ex. : 'Baby', 'Mini-Poussin', 'Poussin', 'Benjamin', 'Minime', 'Cadet', 'Junior', 'Sénior'). Dans le MVP, la validation métier est assurée côté API (express-validator) ; la contrainte SQL sera posée si le temps le permet.
- **historique.auteur (VARCHAR vs FK)** : Le champ auteur est stocké en varchar plutôt qu'en clé étrangère vers utilisateur afin de préserver la traçabilité même en cas de suppression d'un compte admin. C'est un choix de conception intentionnel (audit immuable).
- **Stockage des images (boutique)** : Les images d'articles sont stockées sur le système de fichiers local du VPS dans un répertoire dédié (ex. : /var/www/uploads/boutique/), servi en statique par Nginx via un alias. Seul le chemin relatif est enregistré dans article.image_path. En post-MVP, une migration vers un stockage objet (S3, Scaleway Object Storage) pourra être envisagée.

Index prévus :

- `dossier.statut` — filtre principal du tableau de bord intranet.
- `dossier.date_soumission` — tri chronologique.
- `representant.email` — regroupement visuel fratrie et recherche.
- `utilisateur.email` — lookup d'authentification.
- `historique.dossier_id` — consultation de l'historique par dossier.

Anticipation multi-enfant : Dans le MVP, chaque dossier possède son propre représentant (1:1). Le MLD prévoit néanmoins dès le départ une FK `representant_id` dans `dossier` (relation N:1 prête) afin de faciliter la migration vers un représentant partagé en post-MVP, sans restructuration de schéma.

Prévention des doublons : En prévision du passage en N:1, une contrainte d'unicité `UNIQUE(email)` est posée sur la table `representant` dès le MVP. L'email seul est retenu comme clé de déduplication car un parent peut changer de numéro de téléphone entre l'inscription de deux enfants ; une unicité composite (`email, telephone`) créerait alors un doublon indésirable. Le téléphone reste soumis à une contrainte `NOT NULL` sans participer à l'unicité. Cette approche simplifiera la migration vers un représentant partagé en post-MVP.

4.3 Routes API (périmètre MVP)

Méthode	Route	Description
<i>Routes publiques</i>		
POST	<code>/api/inscription</code>	Soumettre un dossier de pré-inscription <i>Si l'email du représentant existe déjà, le dossier est rattaché au représentant existant (fratrie) au lieu de retourner un 409.</i> <i>Protection double-submit : avant création, l'API vérifie l'existence d'un dossier récent avec le même (email + nom/prénom/date_</i>
GET	<code>/api/infos/horaires</code>	Récupérer les horaires publics
GET	<code>/api/infos/tarifs</code>	Récupérer les tarifs publics
POST	<code>/api/contact</code>	Envoyer un message de contact
POST	<code>/api/auth/login</code>	Connexion administrateur
POST	<code>/api/auth/logout</code>	Déconnexion (invalidation côté client)
<i>Routes protégées (Intranet — rôle admin requis)</i>		
GET	<code>/api/admin/dossiers</code>	Liste des dossiers (filtrable par statut)
GET	<code>/api/admin/dossiers/:id</code>	Détail d'un dossier
PATCH	<code>/api/admin/dossiers/:id/statut</code>	Changer le statut d'un dossier
GET	<code>/api/admin/dossiers/:id/historique</code>	Historique des actions sur un dossier
<i>Routes protégées — Souhaité (si livrées)</i>		
POST	<code>/api/auth/refresh</code>	Renouveler le token JWT (cookie)
PUT	<code>/api/admin/dossiers/:id</code>	Modifier un dossier (champs éditables)
DELETE	<code>/api/admin/dossiers/:id</code>	Supprimer un dossier
PUT	<code>/api/admin/infos/horaires</code>	Modifier les horaires publics
PUT	<code>/api/admin/infos/tarifs</code>	Modifier les tarifs publics
GET	<code>/api/admin/boutique</code>	Liste des articles
POST	<code>/api/admin/boutique</code>	Créer un article
PUT	<code>/api/admin/boutique/:id</code>	Modifier un article
DELETE	<code>/api/admin/boutique/:id</code>	Supprimer un article
GET	<code>/api/boutique</code>	Catalogue public (consultation)

Toute route préfixée `/api/admin` est protégée par le middleware d'authentification JWT et de vérification de rôle. Un appel sans token valide retourne 401 *Unauthorized*; un appel avec un rôle insuffisant retourne 403 *Forbidden*.

4.4 Stratégie de versioning API

Toutes les routes sont préfixées `/api/` sans numéro de version explicite dans le MVP. Ce choix est approprié tant qu'un seul client consomme l'API. Si l'API devait être ouverte à des consommateurs tiers ou subir des évolutions non rétro-compatibles en post-MVP, un préfixe versionné (`/api/v1/`, `/api/v2/`) sera introduit. La migration consistera à ajouter le préfixe `/api/v1/` sur les routes existantes via un middleware de réécriture, sans casser le client actuel.

4.5 Catalogue des codes de réponse HTTP

L'API utilise un ensemble standardisé de codes HTTP. Toute réponse d'erreur suit le format JSON ci-dessous afin d'assurer une gestion homogène côté client :

```
{
  "status": "error",
  "code": 422,
  "message": "Validation échouée.",
  "errors": [
    { "field": "email", "message": "Adresse email invalide." }
  ]
}
```

Code	Signification	Contexte d'usage dans l'API
200	OK	Requête traitée avec succès (GET, PATCH).
201	Created	Ressource créée avec succès (POST <code>/api/inscription</code> , POST <code>/api/admin/boutique</code>).
204	No Content	Suppression réussie (DELETE) ou déconnexion.
400	Bad Request	Requête mal formée (JSON invalide, paramètre manquant).
401	Unauthorized	Token JWT absent, expiré ou invalide.
403	Forbidden	Token valide mais rôle insuffisant pour la ressource demandée.
404	Not Found	Ressource inexistante (dossier, article).
409	Conflict	Conflit d'unicité (ex. : nom d'article déjà utilisé dans la boutique).
422	Unprocessable Entity	Données syntaxiquement correctes mais échouant à la validation métier (champs invalides).
429	Too Many Requests	Seuil de rate-limiting atteint (tentatives d'authentification).
500	Internal Server Error	Erreur serveur non prévue ; le corps ne contient aucun détail technique (stack trace, requête SQL).

Règle de sécurité : Les réponses 401, 403 et 500 ne révèlent jamais la cause technique exacte de l'erreur au client. Le détail est journalisé côté serveur uniquement.

5 SÉCURITÉ

5.1 Synthèse des mesures de sécurité

Mesure	Implémentation	Engagement
Hachage des mots de passe	bcrypt (coût ≥ 12), salage automatique ; aucun stockage en clair.	Garanti
Politique de complexité	Mot de passe admin : ≥ 12 caractères, au moins 1 majuscule, 1 minuscule, 1 chiffre et 1 caractère spécial ; vérification côté serveur (express-validator).	Garanti
Authentification par token	JWT signé (HS256 minimum), durée de vie limitée (ex. : 12h), secret externalisé.	Garanti
Contrôle d'accès côté serveur	Middleware Express vérifiant le token et le rôle sur chaque route protégée.	Garanti
Limitation des tentatives (auth)	Blocage ou ralentissement après 5 échecs / 15 min sur un même identifiant ou IP (express-rate-limit).	Garanti
Limitation anti-spam (contact)	Rate-limit spécifique sur POST /api/contact (ex. : 5 requêtes / 15 min / IP) pour prévenir le spam via le formulaire public.	Garanti
Validation et assainissement des entrées	express-validator sur toutes les routes recevant des données utilisateur ; rejet des caractères dangereux.	Garanti
Requêtes SQL paramétrées	Paramètres positionnels (\$1, \$2) via le driver pg ; aucune concaténation de chaîne.	Garanti
HTTPS obligatoire	Certificat TLS Let's Encrypt sur le VPS ; redirection HTTP $\rightarrow$ HTTPS via Nginx.	Garanti
En-têtes de sécurité HTTP	Helmet.js (X-Content-Type-Options, X-Frame-Options, Strict-Transport-Security, etc.).	Garanti
Variables d'environnement	Secrets (DB, JWT) stockés dans un fichier .env hors du dépôt Git (.gitignore).	Garanti
Journalisation des échecs	Horodatage UTC, IP source, route demandée, code HTTP — en fichier de log ou base.	Garanti
Protection CORS	Origine autorisée restreinte au domaine front-end en production (méthodes : GET, POST, PATCH, PUT, DELETE ; en-têtes : Content-Type, X-CSRF-Token ; credentials : true pour les cookies).	Garanti
Restriction réseau (IP/VPN)	Filtrage par IP source ou tunnel VPN pour l'accès intranet.	Souhaité
Double authentification (2FA)	TOTP ou équivalent pour les comptes admin.	Éventuel
Bannissement IP automatisé	Fail2ban ou équivalent sur les logs d'authentification.	Éventuel

5.2 Gestion des tokens JWT

- **Payload** : Le token contient l'identifiant utilisateur, le rôle et la date d'expiration. Aucune donnée sensible (mot de passe, données personnelles) n'est incluse.
- **Stockage côté client — choix retenu** : Le token JWT est transmis et conservé dans un **cookie** httpOnly, secure et SameSite=Strict, posé par l'API lors de la connexion réussie. Ce choix interdit l'accès au token par le JavaScript client (protection contre le vol via XSS) et assure la persistance entre les rechargements de page sans recourir au localStorage. Le store Pinia conserve uniquement l'état d'authentification dérivé (isAuthenticated, rôle) pour le rendu conditionnel côté client, mais ne stocke jamais le token lui-même.

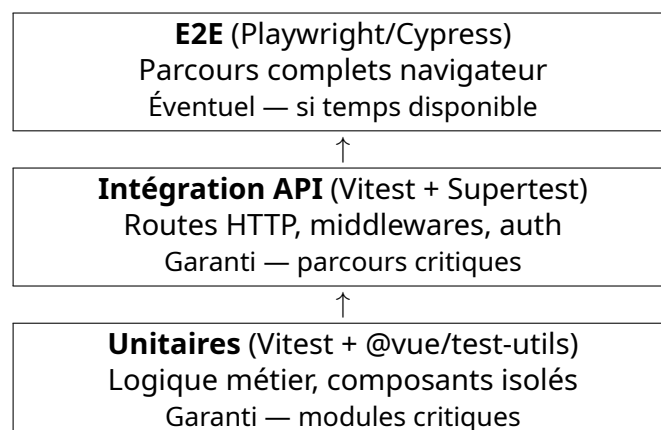
- **Conséquence — protection CSRF obligatoire** : Le cookie étant envoyé automatiquement par le navigateur, une protection CSRF est impérative sur toutes les routes à effet de bord (cf. section OWASP ci-après).
- **Renouvellement** : La durée de vie du token est fixée à **12 heures** afin de couvrir une session de travail complète en soirée sans expiration frustrante. En complément, un endpoint `POST /api/auth/refresh` (**Souhaité**) pourra être ajouté dans le MVP si le temps le permet : il vérifie le token courant encore valide, en émet un nouveau et invalide l'ancien cookie. À défaut, l'utilisateur se reconnecte manuellement à expiration.
- **Invalidation** : La déconnexion supprime le cookie côté serveur (`Set-Cookie` avec `Max-Age=0`). Une blacklist serveur des tokens révoqués n'est pas requise dans le MVP mais reste une évolution possible.

5.3 Protection contre les attaques courantes (OWASP)

- **Injection SQL (A03)** : Requêtes paramétrées exclusivement ; aucune interpolation de chaîne utilisateur.
- **XSS (A03)** : Vue.js échappe par défaut les données injectées dans le DOM ; les directives `v-html` sont interdites sur des données utilisateur non assainies.
- **Broken Access Control (A01)** : Chaque route API vérifie le token et le rôle côté serveur, indépendamment de l'affichage conditionnel côté client.
- **Security Misconfiguration (A05)** : En-têtes Helmet.js, CORS restreint, mode production Express (`NODE_ENV=production`), pas de stack trace exposée.
- **Cryptographic Failures (A02)** : HTTPS en production, bcrypt pour les mots de passe, secrets externalisés.
- **CSRF (Cross-Site Request Forgery)** : Le token JWT étant transmis via un cookie `httpOnly` (cf. section Gestion des tokens JWT), la protection CSRF est obligatoire. Deux mesures combinées : attribut `SameSite=Strict` sur le cookie d'authentification et, en complément, un token CSRF synchronisé (double-submit cookie ou en-tête personnalisé `X-CSRF-Token`) vérifié par le middleware Express sur toutes les routes à effet de bord (POST, PATCH, PUT, DELETE). Le token CSRF est généré côté serveur et transmis au client via un cookie non-`httpOnly` ou un endpoint dédié.

6 STRATÉGIE DE TESTS

6.1 Pyramide de tests retenue



6.2 Configuration de test

- **Framework** : Vitest (compatible Vite, rapide, API Jest-compatible).
- **Exécution** : `npm run test` (unitaires + intégration) ; `npm run test:e2e` (E2E si livré).
- **Couverture** : `@vitest/coverage-v8` — objectif interne de pilotage : $\geq 70\%$ sur les modules critiques (auth, inscription, validation dossier). Cet objectif est un repère de développement, non un critère de recette client.
- **Base de test** : Les tests d'intégration utilisent une base PostgreSQL dédiée (schéma identique, données fictives) ; aucune donnée réelle n'est utilisée pendant les tests.

6.3 Scénarios prioritaires

6.3.1 Tests unitaires

- Validation des champs du formulaire (email, téléphone, date de naissance, longueur).
- Calcul de l'âge à partir de la date de naissance (contrainte 6-80 ans).
- Logique de transition de statut de dossier (transitions autorisées / interdites).
- Génération et vérification de token JWT (signature, expiration, payload).

6.3.2 Tests d'intégration API

- `POST /api/inscription` avec données valides $\rightarrow$ 201 + dossier créé en statut `en_attente`.
- `POST /api/inscription` avec corps JSON incomplet (champs obligatoires absents) $\rightarrow$ 400 ; avec champs présents mais invalides $\rightarrow$ 422 + messages d'erreur lisibles.
- `GET /api/admin/dossiers` sans token $\rightarrow$ 401 Unauthorized.
- `GET /api/admin/dossiers` avec token admin valide $\rightarrow$ 200 + liste de dossiers.
- `PATCH /api/admin/dossiers/:id/statut` avec rôle admin $\rightarrow$ 200 + statut modifié.
- `POST /api/auth/login` avec une 6^e tentative après 5 échecs consécutifs dans une fenêtre de 15 minutes $\rightarrow$ blocage temporaire (429).

6.3.3 Tests E2E (si livrés)

- Parcours inscription : ouverture $\rightarrow$ saisie 3 étapes $\rightarrow$ soumission $\rightarrow$ confirmation.
- Parcours admin : login $\rightarrow$ tableau de bord $\rightarrow$ ouverture dossier $\rightarrow$ validation $\rightarrow$ changement de statut visible.
- Parcours erreurs : soumission formulaire invalide $\rightarrow$ messages d'erreur inline visibles.
- Résolutions de recette : chaque parcours est vérifié sur les cibles définies dans le CDC (375px, 768px, 1280px et plus).

6.4 Données de test (Seeding)

- Un script `seed.sql` génère des dossiers fictifs dans les 4 statuts avec des noms inventés.
- Un compte administrateur de test est créé avec un mot de passe haché connu.
- Les données de seed ne contiennent aucune information réelle de mineurs ou de familles.

7 DÉPLOIEMENT ET EXPLOITATION

7.1 Architecture serveur cible

Composant	Configuration
OS	Ubuntu Server 22.04 LTS ou 24.04 LTS
CPU	1 vCore (minimum)
RAM	2 Go (minimum) — 4 Go recommandés pour une marge confortable en production (Nuxt SSR + Express + PostgreSQL simultanés).
Stockage	20 Go SSD/NVMe (minimum)
Réseau	IPv4 fixe, ports 80/443 ouverts
Reverse proxy	Nginx (terminaison TLS, compression gzip, proxy_pass)
Processus Node.js	PM2 (ou systemd) — redémarrage automatique, gestion des logs
Base de données	PostgreSQL 16 sur le même VPS
Certificat TLS	Let's Encrypt via Certbot (renouvellement cron toutes les 12h)
Pare-feu	UFW — ports 22 (SSH), 80, 443 ouverts ; tout le reste fermé
Swap	1 Go de swap configuré (<code>fallocate + mkswap</code>) — filet de sécurité mémoire
Surveillance mémoire	Alerte si RAM libre < 200 Mo (script cron ou outil léger type <code>monit</code>)

7.2 Procédure de déploiement (mode nominal — Garanti)

Le déploiement initial et les mises à jour seront réalisés manuellement via SSH selon la procédure documentée ci-dessous. Cette procédure sera transcrite dans le guide d'exploitation livré au club.

1. Connexion SSH au VPS (clé SSH recommandée, mot de passe désactivé).
2. Pull du code source depuis le dépôt Git distant.
3. Installation des dépendances : `npm install --production` (API et client).
4. Build du front-end : `npm run build` (Nuxt génère le bundle de production).
5. Migration de la base si nécessaire : exécution des scripts SQL de migration.
6. Redémarrage des processus : `pm2 restart all` (ou équivalent systemd).
7. Vérification : accès au site, test de la page d'accueil et d'une route API protégée.

Rollback : En cas de problème après déploiement, le code peut être ramené à la version précédente via `git checkout` et la base restaurée depuis le dernier dump `pg_dump`.

7.3 Configuration Nginx (cible)

```
server {
    listen 443 ssl http2;
    server_name uspvjudo.fr www.uspvjudo.fr;

    ssl_certificate      /etc/letsencrypt/live/uspvjudo.fr/fullchain.pem;
    ssl_certificate_key  /etc/letsencrypt/live/uspvjudo.fr/privkey.pem;

    # Proxy vers le serveur Nuxt (SSR + pages publiques)
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # Proxy vers l'API Express
    location /api/ {
        proxy_pass http://127.0.0.1:4000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # En-têtes de sécurité
    add_header X-Content-Type-Options nosniff;
    add_header X-Frame-Options DENY;
    add_header Strict-Transport-Security
        "max-age=31536000; includeSubDomains" always;
    # CSP (Content-Security-Policy) à ajuster selon les besoins :
    # add_header Content-Security-Policy "default-src 'self'; ..." always;

    # Fichiers statiques boutique (ajouté si module boutique livré) :
    # location /uploads/boutique/ {
    #     alias /var/www/uploads/boutique/;
    #     expires 30d;
    #     add_header Cache-Control "public";
    # }
}

server {
    listen 80;
    server_name uspvjudo.fr www.uspvjudo.fr;
    return 301 https://$host$request_uri;
}
```

Le nom de domaine `uspvjudo.fr` est donné à titre d'exemple ; le domaine effectif sera confirmé par l'association. Sur les versions récentes de Nginx ($\geq 1.25.1$), la directive `http2 on`; remplace `listen 443 ssl http2`; ; la syntaxe sera adaptée à la version installée sur le VPS.

7.4 Variables d'environnement

Le fichier `.env` (jamais commité dans Git) contient les paramètres sensibles suivants :

Variable	Description
DATABASE_URL	Chaîne de connexion PostgreSQL (format : <code>postgresql://user:pass@host:5432/dbname</code>).
JWT_SECRET	Secret de signature des tokens JWT (chaîne aléatoire ≥ 64 caractères).
JWT_EXPIRES_IN	Durée de vie du token (ex. : 12h).
NODE_ENV	production en exploitation, development en local.
PORT_API	Port d'écoute de l'API Express (ex. : 4000).
PORT_CLIENT	Port d'écoute du serveur Nuxt (ex. : 3000).
CORS_ORIGIN	Origine autorisée pour les requêtes CORS (domaine front-end).
RATE_LIMIT_WINDOW_MS	Fenêtre de limitation des tentatives (ex. : 900000 = 15 min).
RATE_LIMIT_MAX	Nombre maximal de tentatives dans la fenêtre (ex. : 5).
CSRF_SECRET	Secret de signature des tokens CSRF (chaîne aléatoire ≥ 32 caractères) ; utilisé par le middleware de protection CSRF sur les routes à effet de bord.

Un fichier `.env.example` est fourni dans le dépôt avec les clés sans valeurs sensibles, servant de modèle pour la configuration.

8 SAUVEGARDE ET RESTAURATION

8.1 Dispositif de sauvegarde

- **Fréquence** : Sauvegarde quotidienne automatisée via tâche cron (ex. : chaque nuit à 03h00).
- **Outil** : `pg_dump` en mode custom (`-Fc`) pour un dump compressé et restaurable sélectivement.
- **Chiffrement** : Le dump est chiffré avant transfert (ex. : GPG ou chiffrement intégré `rclone`).
- **Externalisation** : Transfert automatique vers un stockage objet distant (OVH Object Storage, Scaleway Object Storage ou Backblaze B2) via `rclone`.
- **Rétention** : Conservation des 30 dernières sauvegardes journalières (rotation automatique).
- **Coût** : Négligeable pour les volumes concernés (quelques Mo/jour) — inclus dans le budget annuel estimé.

8.2 Script de sauvegarde (cible)

```
#!/bin/bash
# backup-db.sh - Sauvegarde quotidienne PostgreSQL
set -euo pipefail

TIMESTAMP=$(date +%Y%m%d_%H%M%S)
DUMP_FILE="/var/backups/uspv/uspv_${TIMESTAMP}.dump"
REMOTE="backup-remote:uspv-backups/"

# 1. Dump PostgreSQL
pg_dump -Fc -U uspv_backup -d uspv_judo -f "$DUMP_FILE"
```

```
# 2. Chiffrement
gpg --batch --yes --symmetric \
    --passphrase-file /root/.backup-passphrase \
    "$DUMP_FILE"
rm "$DUMP_FILE"

# 3. Transfert distant
rclone copy "${DUMP_FILE}.gpg" "$REMOTE"

# 4. Rotation locale (30 jours)
find /var/backups/uspv/ -name "*.gpg" -mtime +30 -delete

echo "[$(date)] Sauvegarde OK: ${DUMP_FILE}.gpg"
```

8.3 Procédure de restauration

1. Récupérer le dump chiffré depuis le stockage distant : `rclone copy backup-remote:uspv-backups/fichier.dump.gpg /tmp/`.
2. Déchiffrer : `gpg --decrypt --output /tmp/fichier.dump /tmp/fichier.dump.gpg`.
3. Restaurer dans une base de test : `pg_restore -U uspv_backup -d uspv_judo_test /tmp/fichier.dump`.
4. Vérifier l'intégrité des données restaurées (comptage de dossiers, cohérence des statuts).
5. Si la vérification est concluante, restaurer en production (avec arrêt préalable de l'application).

8.4 Test de restauration

- **Fréquence** : Un test de restauration est réalisé une fois par mois sur une base de test distincte.
- **Vérification** : Le test confirme que le dump est lisible, que le schéma est intact et que les données sont cohérentes.
- **Traçabilité** : La date et le résultat du test sont consignés dans un fichier de suivi d'exploitation.

9 JOURNALISATION ET SURVEILLANCE

9.1 Bibliothèque de logging

Les logs applicatifs seront produits via **pino**, bibliothèque de logging structuré pour Node.js. Pino est retenu pour sa performance (logging asynchrone, overhead minimal), son format JSON natif (compatible avec des outils d'analyse de logs) et sa légèreté par rapport à Winston. En développement, le module `pino-pretty` formatera les logs pour la lisibilité en console.

9.2 Logs applicatifs (Garanti)

- **Format** : Chaque entrée de log contient au minimum : horodatage UTC, niveau (info/warn/error), IP source, méthode HTTP, route, code de réponse, message. Les logs sont émis au format JSON structuré via pino.
- **Événements journalisés** :
 - Tentatives d'authentification (succès et échecs).
 - Accès refusés (401, 403) aux routes protégées.
 - Erreurs serveur (500) avec contexte suffisant pour le diagnostic.
 - Actions d'administration sur les dossiers (changement de statut, auteur, horodatage).
- **Stockage** : Fichiers de log sur le VPS, gérés par PM2 ou un mécanisme de rotation (logrotate).
- **Confidentialité** : Les mots de passe, tokens et données personnelles ne sont jamais écrits dans les logs.

9.3 Surveillance (Souhaité / Événuel)

- **Souhaité** : Vérification manuelle régulière des logs ; alerte email en cas d'erreur 500 récurrente (script cron simple).
- **Événuel** : Intégration de Fail2ban pour le bannissement automatique d'IP après échecs d'authentification répétés sur les logs Nginx/applicatifs.

10 INSTRUMENTATION ET MESURE D'AUDIENCE

10.1 Choix de l'outil (Souhaité)

- **Recommandation** : Plausible Analytics (auto-hébergé ou cloud) ou Matomo en configuration anonymisée.
- **Justification** : Outils respectueux de la vie privée, exemptés de consentement cookie dans la plupart des configurations, conformes RGPD.
- **Alternative** : Si l'association le préfère, Google Analytics avec bandeau de consentement cookie conforme CNIL.

10.2 Événements applicatifs (Souhaité)

Si la brique d'instrumentation est livrée, les événements suivants seront tracés pour alimenter les KPI définis dans le cahier des charges :

- `inscription_start` — Le parent ouvre le formulaire d'inscription.
- `inscription_step_N` — Le parent atteint l'étape N du parcours.
- `inscription_submit` — Le formulaire est soumis avec succès.
- `inscription_error` — Une erreur de validation bloque la soumission.
- `admin_dossier_open` — Un administrateur ouvre un dossier.

- `admin_dossier_validate` — Un administrateur valide un dossier.

Ces événements permettent de calculer le taux de conversion, le taux d'abandon et les temps moyens définis dans les KPI post-production.

11 ENVIRONNEMENTS

Environnement	Usage	Base de données	Engagement
Développement local	Développement quotidien sur la machine du stagiaire.	PostgreSQL local (données fictives).	Garanti
Test / Préproduction	Recette fonctionnelle, démonstration aux validateurs.	Copie du schéma avec données fictives.	Souhaité
Production	Application accessible au public et au bureau du club.	Base de production (données réelles).	Garanti

Isolation : Les bases de données de chaque environnement sont strictement séparées. Aucune donnée réelle n'est jamais utilisée en développement ou en test.

12 PERFORMANCE ET DIMENSIONNEMENT

12.1 Charge attendue

Le club compte de l'ordre de 80 à 120 licenciés par saison. Le pic d'activité se situe en septembre-octobre (période d'inscriptions). La charge attendue reste très modeste :

- **Visiteurs simultanés :** 5 à 15 en pic (rentrée sportive).
- **Dossiers d'inscription :** 50 à 120 par saison.
- **Sessions admin simultanées :** 1 à 3 maximum.

12.2 Optimisations prévues

- **SSR Nuxt :** Les pages publiques sont rendues côté serveur et mises en cache par Nginx (cache statique des assets, compression gzip/brotli).
- **Images :** Compression et formats modernes (WebP) pour les visuels du club et du catalogue boutique.
- **Requêtes SQL :** Index ciblés sur les colonnes de filtre et de tri (cf. section Modèle de données).
- **Bundle front-end :** Code-splitting automatique par Nuxt (chargement des pages à la demande).

Compte tenu de la charge attendue, un VPS 1 vCore / 2 Go RAM est suffisant. En production, PostgreSQL 16, l'API Express et le serveur Nuxt SSR tournent simultanément ; un swap de 1 Go est configuré comme filet de sécurité et un monitoring mémoire simple (cron ou `monit`) alerte si la RAM libre descend sous 200 Mo. Aucune infrastructure de mise à l'échelle (load balancer, cluster) n'est nécessaire.

13 CI/CD (ÉVENTUEL)

Si le temps le permet, une pipeline CI/CD légère sera configurée via GitHub Actions :

1. **Trigger** : Push sur la branche `main` ou ouverture de Pull Request.
2. **Étapes** :
 - Installation des dépendances (`npm ci`).
 - Lint (vérification du style de code).
 - Tests unitaires et d'intégration (`npm run test`).
 - Build du front-end (`npm run build`).
3. **Déploiement automatique (if applicable)** : Push sur `main` → déploiement SSH sur le VPS via GitHub Actions.

Cette brique est classée **Éventuel** : le déploiement manuel reste le mode nominal garanti.

14 SYNTHÈSE DES DÉPENDANCES NPM PRINCIPALES

Paquet	Couche	Rôle
nuxt	Client	Meta-framework Vue.js (SSR/CSR, routing, build).
vue	Client	Bibliothèque réactive (composants, Composition API).
pinia	Client	Gestion d'état centralisée.
express	API	Framework HTTP pour l'API REST.
pg	API	Driver PostgreSQL natif pour Node.js.
jsonwebtoken	API	Création et vérification des tokens JWT.
bcryptjs	API	Hachage et vérification des mots de passe.
express-validator	API	Validation et assainissement des entrées.
express-rate-limit	API	Limitation des tentatives par IP/identifiant.
cors	API	Gestion des en-têtes CORS.
helmet	API	En-têtes de sécurité HTTP.
pino	API	Logging structuré JSON, performant et asynchrone.
pino-pretty	API (dev)	Formatage lisible des logs pino en développement.
dotenv	API	Chargement des variables d'environnement depuis <code>.env</code> .
cookie-parser	API	Lecture des cookies (nécessaire pour le JWT en cookie httpOnly).
csrf-csrf	API	Middleware de protection CSRF double-submit cookie (génère et vérifie le token CSRF).
vitest	Dev	Framework de tests (unitaires + intégration).
@vue/test-utils	Dev	Tests de composants Vue.
supertest	Dev	Tests HTTP contre l'API Express.
@vitest/coverage-v8	Dev	Rapport de couverture de code.
playwright (ou cypress)	Dev	Tests E2E navigateur (si livrés).

Les versions exactes seront verrouillées dans les fichiers `package-lock.json` respectifs et mises à jour en cas d'alerte de sécurité.

15 GLOSSAIRE

Terme	Définition
API REST	Interface de programmation basée sur le protocole HTTP et les conventions REST (GET, POST, PUT, DELETE).
bcrypt	Algorithme de hachage de mots de passe avec salage automatique intégré.
CI/CD	Intégration Continue / Déploiement Continu — automatisation des tests et du déploiement.
CORS	Cross-Origin Resource Sharing — mécanisme de contrôle des requêtes entre domaines différents.
CSR	Client-Side Rendering — le HTML est généré dans le navigateur par JavaScript.
DAT	Document d'Architecture Technique.
JWT	JSON Web Token — standard de token d'authentification signé et vérifiable.
MCD / MLD	Modèle Conceptuel / Logique de Données — schémas de conception de la base de données.
MVP	Minimum Viable Product — périmètre minimal livrable garanti.
Nginx	Serveur web / reverse proxy haute performance.
ORM	Object-Relational Mapping — couche d'abstraction entre le code applicatif et la base SQL.
PM2	Gestionnaire de processus Node.js en production (redémarrage, logs, monitoring).
SPA	Single Page Application — application web chargée une seule fois, navigation sans rechargement.
SSR	Server-Side Rendering — le HTML est généré côté serveur avant envoi au navigateur.
TLS / HTTPS	Transport Layer Security — chiffrement des échanges HTTP.
VPS	Virtual Private Server — serveur virtuel loué chez un hébergeur.